

EventHub

Event Management & Ticketing Platform

Full-Stack Web Application Project
React | Node.js | Supabase | Razorpay

Developed by: Satyam Revgade
Computer Engineering | VIT Mumbai
Year: 2026

This report covers the design, development, and deployment of EventHub - a full-stack SaaS platform for discovering events, handling registrations, processing payments via Razorpay, and performing QR-based check-ins. It includes system architecture, feature set, technical decisions, challenges encountered, and planned improvements.

1. Problem Statement

Managing event registrations today is unnecessarily fragmented. Organisers often juggle separate tools for creating event listings, collecting payments, sending confirmation emails, and validating attendees at the venue. For smaller events and communities, this friction means either paying for expensive SaaS platforms or resorting to manual spreadsheet-based tracking - both with obvious shortcomings.

EventHub solves this by combining the full registration lifecycle into a single self-hosted application: event discovery for attendees, Razorpay-powered payments, automatic QR ticket generation, and a real-time admin dashboard covering check-ins, revenue, and attendee management.

2. Objectives

- * Build a working end-to-end registration flow - from browsing events to receiving a scannable ticket.
- * Integrate Razorpay for both paid and free event registrations, with server-side payment verification.
- * Provide admins with a self-contained dashboard: event CRUD, revenue analytics, attendee lists, and live QR scanning.
- * Deploy on free-tier infrastructure (Vercel + Render + Supabase) with no ongoing cost.
- * Make the UI fully responsive and support dark mode out of the box.

3. Features

For Users

- * Browse and filter events by category (Technology, Design, Music, etc.)
- * View full event details - venue, date, capacity, price, description
- * Register for free events in one click
- * Pay for paid events via Razorpay checkout modal (test mode)
- * Receive a unique QR-coded digital ticket after successful registration
- * View all registered tickets under My Tickets with QR image display
- * Dark mode toggle with persistent preference

For Admins

- * Secure admin registration and login (separate from user accounts)
- * Dashboard with KPI cards: total events, registrations, revenue, check-ins
- * Create, edit, and delete events with a modal form
- * QR ticket scanner: camera-based scan or manual JSON entry for check-in validation
- * Revenue breakdown per event with confirmed registration counts
- * Attendee list per event showing check-in status and timestamps
- * User management panel with the ability to create additional admin accounts

4. Tech Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	UI framework and build tool
	Tailwind CSS	Utility-first styling with dark mode support
	React Router v6	Client-side routing (HashRouter for Vercel)
	Axios	HTTP client with retry logic for cold starts

Layer	Technology	Purpose
Backend	Lucide React	Icon library
	html5-qrcode	Camera-based QR code scanning
	Node.js + Express	REST API server
	Supabase JS SDK	Database client (PostgreSQL via Supabase)
	jsonwebtoken	Stateless JWT authentication
	bcryptjs	Password hashing (cost factor 12)
	Razorpay SDK	Payment order creation and signature verification
	qrcode	QR code image generation (base64 PNG)
	helmet + rate-limit	HTTP security headers and request throttling
	Supabase (PostgreSQL)	Hosted relational database
Hosting	Vercel / Render	Frontend + backend deployment

5. System Architecture

The application follows a three-tier architecture: a React SPA as the presentation layer, an Express REST API as the business logic layer, and Supabase (PostgreSQL) as the data layer. The frontend communicates with the backend exclusively over JSON REST calls via Axios. The backend uses the Supabase service-role key, bypassing Row-Level Security for simplicity.

Database Schema

Table	Key Columns	Notes
users	id, name, email, password_hash, role	role: 'user' or 'admin'
events	id, title, event_date, venue, capacity, price, status, created_at	status: draft/published/ongoing/completed/cancelled
registrations	id, user_id, event_id, status, amount_paid, razorpay_order_id	UNIQUE(user_id, event_id) prevents duplicates
tickets	id, registration_id, qr_code, qr_data, status, checked_in_at, checked_in_by	Foreign Key to registrations (registration_id) + checked_in_by

Payment & Ticket Flow

Step	Action
1	User selects a paid event and clicks Register Now
2	Frontend calls POST /api/payments/create-order - backend creates a Razorpay order
3	Razorpay checkout modal opens; user completes payment
4	Razorpay returns payment_id, order_id, and signature to the frontend

Step	Action
5	Frontend POSTs these to /api/payments/verify
6	Backend verifies HMAC-SHA256 signature; if valid, marks registration confirmed
7	Backend generates QR code (JSON payload base64 PNG) and inserts a ticket row
8	User sees the QR ticket on /my-tickets
9	At venue, admin selects event on /admin/scanner and scans QR - backend validates and marks ticket used

6. Challenges & Solutions

Supabase Ambiguous Foreign Key (PostgREST)

Problem: The tickets table has two foreign keys to users - one for user_id and one for checked_in_by. PostgREST's auto-join syntax threw an ambiguous relationship error that was silently caught and returned as Ticket not found, causing the QR scanner to fail on every valid ticket.

Solution: Fixed by using PostgREST's explicit FK hint syntax: users!user_id(name, email) in all three ticket endpoints, telling PostgREST exactly which relationship to follow.

Render Backend Cold Starts

Problem: Render's free tier spins down the server after ~15 minutes of inactivity. The first request after sleep can take 30-60 seconds, during which the frontend showed a broken empty state with no feedback.

Solution: Built a useServerStatus hook that detects ERR_NETWORK failures, shows a Server is waking up banner, and retries with exponential backoff - up to 10 attempts, capped at 15s delay. UI restores automatically once the server responds.

Role Constraint Violation on Registration

Problem: The DB enforces role IN ('user', 'admin'). The frontend sends role: organizer for admin registrations, which violated the CHECK constraint and returned a 500 error.

Solution: Added a mapping in auth.js: const userRole = role === 'organizer' ? 'admin' : 'user'. Frontend role labels are purely UI-level; the DB always receives a valid value.

Dark Mode Styling

Problem: Tailwind's class-based dark mode requires explicit dark: variants on every component. Initial implementation used hardcoded light-mode classes only, making most text invisible on dark backgrounds.

Solution: Audited all pages and components, adding dark: variants throughout. CSS variables handle global token overrides; explicit Tailwind classes handle component-level colours.

QR Scanner Without Event Scoping

Problem: The original scanner had no event selection - it could validate any ticket against any event, making cross-event check-ins possible.

Solution: Added an event dropdown to the scanner UI. The event_id is sent alongside qr_data to the validate endpoint, which enforces that the ticket's eventId matches before check-in.

7. Security

Control	Implementation
Password Hashing	bcrypt with cost factor 12. Plain passwords never stored.
JWT Auth	Signed tokens, configurable expiry. Every protected endpoint verifies token and role.
Payment Verification	Razorpay HMAC-SHA256 signature verified server-side before any ticket is issued.
Rate Limiting	100 req/15min general; 20 req/15min on auth endpoints.
HTTP Headers	helmet.js sets HSTS, X-Frame-Options, X-Content-Type-Options on all responses.
CORS	Restricted to the configured FRONTEND_URL env variable only.
Admin Scoping	JWT + role admin double-check. Dashboard/revenue data scoped to requesting admin's events.

8. Future Improvements

Email Notifications

Send confirmation emails with the QR ticket attached after registration, plus event reminder emails 24 hours before the event.

Waitlist System

When an event is full, users join a waitlist. On cancellation, the next person is notified and given a time window to complete registration.

Row-Level Security (RLS)

Currently the service-role key bypasses RLS. Switching to scoped keys with proper RLS policies would improve the security model for multi-admin setups.

Event Analytics

Per-event charts: registration trend over time, check-in rate by hour, and payment funnel drop-off analysis.

Ticket Cancellations & Refunds

Let users cancel registrations. For paid events, trigger Razorpay's refund API and update ticket status, with an admin-configurable cancellation deadline.

OAuth Login

Add Google and GitHub OAuth via Supabase Auth so users don't need to manage a password.

Progressive Web App (PWA)

Add a service worker and manifest so users can install EventHub on mobile and access their QR tickets offline.

9. Conclusion

EventHub demonstrates a complete, deployment-ready full-stack application built with a modern JavaScript stack. The project covers the full spectrum of web development: relational database design, RESTful API architecture, JWT-based authentication, third-party payment integration with server-side verification, real-time QR-based check-in, and a responsive React frontend with dark mode support.

Beyond the happy path, the project handles real-world edge cases - duplicate registrations, sold-out events, ambiguous database joins, cold start latency, and payment signature forgery attempts - making it a strong reference for production-grade web application patterns.